

The Heatflask Visualization

A periodic sampling of recorded motion, and its evaluation on a GPU

Efrem Rensi

Draft — 22 September 2026

Draft status. Second draft, not for circulation. Part I is the mathematics; Part II is the WebGL2 realization. Licensing for the text is undecided — see Colophon.

Abstract

Heatflask animates large collections of recorded GPS activities by drawing, for each activity, the athlete’s position at a periodic sequence of past instants. The resulting figure has a property that is easy to observe and worth stating precisely: the *set* of marks is stationary in time while each individual mark moves along the track. This note derives that property, extracts from it an indexing of the marks that does not depend on time, and shows that this indexing is exactly what permits the entire animation to be evaluated in a vertex shader with one scalar of per-frame input. Part II describes that implementation, including a resampling scheme that replaces a search over irregular data with two array lookups, and gives error bounds for it.

Notation

symbol	meaning	units
P	position along a recording, as a function of time since its start	world pixels
D	duration of a recording	s
s	absolute start time of a recording	s (epoch)
u	the animation clock	s (absolute, activity-scale)
τ	clock rate: activity-seconds per real second	s/s
T	<i>sparsity</i> : spacing between successive marks	s
θ	time since the start of a recording	s
ϕ	<i>phase</i> , $u \bmod T$	s
α	<i>anchor</i> of a recording, $s \bmod T$	s
k	index of a mark, counted back from the leading one	—
h	resampling step	s
ν	validity flag of a resampled node	$\{0, 1\}$
λ, φ	longitude, latitude	rad

Two of these are user-facing dials, and they are independent: T controls how many marks appear, τ controls how fast they move. Defaults are $T = 60$ s and $\tau = 30$.

Part I — The model

1. The recording

An *activity* is a finite sequence of timestamped positions,

$$(t_0, P_0), (t_1, P_1), \dots, (t_{n-1}, P_{n-1}), \quad 0 = t_0 < t_1 < \dots < t_{n-1} = D.$$

The times are measured from the start of the recording and are *irregular*: a GPS unit samples when it can. Positions are in Web Mercator world pixels — the whole Earth mapped to a 256×256 square, so that at zoom z the map scale is 2^z and coordinates transform by a similarity. Nothing below depends on this choice; any affine coordinate system will do.

Extend the samples to a function by linear interpolation. For $t \in [t_i, t_{i+1}]$,

$$P(t) = P_i + \frac{t - t_i}{t_{i+1} - t_i} (P_{i+1} - P_i),$$

so $P : [0, D] \rightarrow \mathbb{R}^2$ is continuous, piecewise affine, and agrees with the recording at every sample. It is the athlete's position at time θ after they started, as far as the recording knows.

The activity occupies the absolute interval $[s, s + D]$.

2. The visualization

Let u be the animation clock, an absolute time advancing at rate τ : $du/dt_{\text{real}} = \tau$.

Definition 1. Fix a sparsity $T > 0$. At clock u , the activity (s, D, P) displays the finite set of marks

$$M(u) = \{ P(u - jT - s) : j \in \mathbb{Z}, 0 \leq u - jT - s \leq D \}.$$

In words: the athlete's position now, and at T seconds ago, and $2T$ seconds ago, and so on, for as far back as the recording extends. Every mark is a past self. All of them move at once, and every activity in view is doing this simultaneously against its own start time.

3. Periodicity

Proposition 1. $M(u + T) = M(u)$ for all u .

Proof. Substituting $u \mapsto u + T$ and reindexing $j \mapsto j + 1$ leaves both the expression $u - jT - s$ and the constraint unchanged. $\square$

This is the visual signature of the whole design, and it is worth saying what it does and does not assert. The set of *occupied positions* repeats with period T ; the marks themselves do not return. Over one period each mark advances precisely into its predecessor's place. What the eye sees is a stationary pattern of beads threaded on a fixed string, with the beads sliding along it — not a pattern that translates.

The population turns over at one mark per period. Writing $r = (u - s) \bmod T$, the occupied times are $r, r + T, r + 2T, \dots$ up to D ; as r increases through $[0, T)$ every mark advances, the leading one eventually passes D and disappears, and when r wraps a new mark appears at $\theta = 0$. Each mark therefore traverses the entire recording, taking D activity-seconds — that is D/τ seconds of real time — from trailhead to finish. The count is not constant:

$$|M(u)| = \left\lfloor \frac{D-r}{T} \right\rfloor + 1 \in \left\{ \left\lfloor \frac{D}{T} \right\rfloor, \left\lfloor \frac{D}{T} \right\rfloor + 1 \right\},$$

oscillating by one within each period. Periodicity of the *set* and constancy of its *cardinality* are different claims, and only the first holds.

Proposition 2 (spacing reads as pace). Consecutive marks are separated by $\|P(\theta + T) - P(\theta)\|$. Where P is differentiable,

$$\|P(\theta + T) - P(\theta)\| = T\|\dot{P}(\theta)\| + O(T^2).$$

So the bead spacing is, to first order, T times speed: the marks bunch where the athlete was slow and stretch out where they were fast. This is the reason the figure is informative rather than decorative, and it

is also the reason resampling must preserve *timing* and not merely shape — see §14.

4. Reindexing: the substitution that makes this computable

Definition 1 indexes marks by j , an absolute count of periods. That index is useless to a renderer: it increases without bound, and the set of admissible j slides forward as u advances, so no mark keeps its name for longer than a period. We remove both defects with one substitution.

Let $\theta = u - s$ be the activity-time of the leading mark, and write

$$\theta = qT + r, \quad q = \left\lfloor \frac{\theta}{T} \right\rfloor, \quad r = \theta \bmod T \in [0, T).$$

Put $k = q - j$. Then $u - jT - s = \theta - jT = r + kT$, and the constraint $0 \leq \theta - jT \leq D$ becomes $0 \leq r + kT \leq D$. Since $r \geq 0$ and $T > 0$, the lower bound forces $k \geq 0$.

Definition 2. The k -th mark of the activity sits at activity-time

$$\theta_k = ((u - s) \bmod T) + kT, \quad k = 0, 1, 2, \dots$$

and is displayed exactly when $\theta_k \leq D$.

Proposition 3 (the index set is time-independent). For every u , the set of displayed indices is contained in

$$K = \{0, 1, \dots, \lfloor D/T \rfloor\},$$

which does not depend on u .

Proof. k is displayed iff $r + kT \leq D$, i.e. $k \leq (D - r)/T$. As $r \in [0, T)$ we have $(D - r)/T \leq D/T$, so $k \leq \lfloor (D - r)/T \rfloor \leq \lfloor D/T \rfloor$. $\square$

Proposition 3 is the load-bearing result of this note. Under the j -indexing, “the marks” are a moving window over an unbounded set and there is nothing stable to hand a renderer. Under the k -indexing, the pair (activity, k) is a *permanent address*: it names the same mark for the entire life of the figure, it ranges over a set fixed by D and T alone, and $|K|$ is known the moment the activity is loaded. A renderer can therefore enumerate every mark that will ever exist, once, and then never revisit the question. Marks not currently displayed are not removed from the enumeration; they simply fail the test $\theta_k \leq D$ and draw nothing.

5. The phase decomposition

Definition 2 still contains $u - s$, which couples the clock to each activity. The coupling is removable.

Proposition 4. With $\phi = u \bmod T$ and $\alpha = s \bmod T$,

$$(u - s) \bmod T = (\phi - \alpha) \bmod T.$$

Proof. Reduction mod T is a homomorphism of additive groups $\mathbb{R} \rightarrow \mathbb{R}/T\mathbb{Z}$, so it carries $u - s$ to $\bar{u} - \bar{s}$. $\square$

Trivial as mathematics, decisive as engineering. Substituting into Definition 2,

$$\theta_k = (\phi - \alpha) \bmod T + kT,$$

and the variables separate: α belongs to the activity and k to the mark, while ϕ — one scalar, shared by every activity on the map — is the only thing that depends on the moment. Three consequences follow.

(a) A frame is one number. Advancing the animation means recomputing ϕ and nothing else. Whatever is loaded — one activity or a thousand, a thousand GPS points or a million — the per-frame state of the animation is a single real number in $[0, T)$.

(b) **The number is small, which matters in single precision.** The clock runs at absolute scale: the implementation initializes u to (epoch seconds) $\times \tau$, which with the default $\tau = 30$ is of order 10^{11} . A binary32 float carries about 7 significant decimal digits, so at that magnitude the representable spacing is thousands of seconds and the animation would not move at all. Reducing mod T *before* narrowing to single precision bounds the transmitted value by T , and the difficulty disappears. The reduction is performed in double precision on the host; only ϕ is narrowed.

(c) **The phase is anchored to the recording, not to the viewport.** $\alpha = s \bmod T$ depends on the activity's start time alone. Nothing in Definition 2 refers to the map's position, zoom, or bearing, so panning cannot alter where a mark is. This is worth flagging because the 2020 renderer did not have the property: it anchored the phase to the first *visible* segment, so every pan re-phased the trail and the marks visibly jumped. Under the present model that failure is not a bug that has been fixed but a statement that cannot be formulated — the view is not an argument of θ_k .

6. Resampling: replacing a search with arithmetic

Evaluating $P(\theta)$ requires locating θ among the irregular $t_0 < \dots < t_{n-1}$: a binary search, $O(\log n)$, with a data-dependent branch at every step. Part II explains why that is the wrong shape for the hardware. Here we record what replaces it.

Fix a step $h > 0$ and let $\tilde{P}$ be the piecewise-affine interpolant of P on the *uniform* mesh $\{0, h, 2h, \dots\}$:

$$\tilde{P}(\theta) = (1 - \lambda) P(ih) + \lambda P((i+1)h), \quad i = \left\lfloor \frac{\theta}{h} \right\rfloor, \quad \lambda = \frac{\theta}{h} - i.$$

Evaluation is now one multiplication, one floor, two array reads and one interpolation — constant time, no branches, no search. The mesh is computed once per activity, when it is loaded.

Fitting the mesh to the recording. h is not a global constant. Given a ceiling $h_{\max}$ (2 s in the implementation), each activity takes

$$m = \min\left(\left\lceil \frac{D}{h_{\max}} \right\rceil, 2^{14}\right), \quad h = \frac{D}{m},$$

so that $mh = D$ *exactly*, and $h \leq h_{\max}$ for any recording shorter than $2^{14}h_{\max}$ — just over nine hours at the default. Past that the step lengthens rather than the node count growing, so no single activity can claim more than $2^{14} + 2$ texels. The last node lands on the end of the recording. The alternative — a fixed grid $0, h_{\max}, 2h_{\max}, \dots$ — leaves a final partial step whose upper bracket lies beyond D , where there is no data; a mark in that interval cannot be evaluated and vanishes. The symptom is that every activity goes dark over its last few seconds, which is precisely what the fixed grid produced before September 2026. Fitting the mesh to D removes the case rather than handling it, and costs no extra storage.

One node is stored past the end, carrying $\nu = 0$, so that the final real step still has an upper bracket to read. It is never interpolated against: a mark with $\theta_k > D$ has already failed Definition 2.

7. How much accuracy this costs

$\tilde{P}$ is a linear interpolation of a linear interpolation, and the composition is not the identity. The error is small, structured, and worth bounding rather than hand-waving.

Proposition 5 (exactness away from corners). If the open interval $(ih, (i+1)h)$ contains no recorded time t_j , then $\tilde{P} = P$ on $[ih, (i+1)h]$.

Proof. Both endpoints lie in one linear piece of P , on which P is affine. The affine interpolant of an affine function through two of its points is that function. $\square$

So error is not distributed along the track; it is supported entirely on the mesh intervals that straddle a recorded point, and vanishes identically everywhere else. With $h \leq 2$ s against recordings sampled at

around 1 Hz, most intervals straddle one or two points, so this is a statement about where the error concentrates rather than a proof that it is rare. What matters is that it is *bounded by the athlete's motion over one step*.

Fix such an interval $I = [ih, (i+1)h]$, write $A = P(ih)$ and $B = P((i+1)h)$, and let

$$L = \int_I \|\dot{P}\| d\theta \quad (\text{arc length}), \quad c = \|B - A\| \quad (\text{chord length}),$$

noting $c \leq L \leq vh$ where $v = \sup_I \|\dot{P}\|$.

Proposition 6 (the corner cut). Let $\hat{n}$ be a unit normal to $B - A$. For every $\theta \in I$,

$$|\langle P(\theta) - \tilde{P}(\theta), \hat{n} \rangle| \leq \frac{1}{2} \sqrt{L^2 - c^2},$$

and the bound is attained.

Proof. Rectifiability gives $\|P(\theta) - A\| + \|P(\theta) - B\| \leq L$, so $P(\theta)$ lies in the closed ellipse with foci A, B and major axis L , whose semi-minor axis is $\frac{1}{2}\sqrt{L^2 - c^2}$. That ellipse's extent normal to AB is exactly its semi-minor axis, and $\tilde{P}(\theta)$ lies on the segment AB , where the normal component is zero.

For attainment, take a symmetric right-angle corner: legs of length ℓ traversed at constant speed, so $L = 2\ell$ and $c = \ell\sqrt{2}$. At the corner time, P is at the vertex and $\tilde{P}$ at the chord's midpoint; the distance between them is $\ell/\sqrt{2} = \frac{1}{2}\sqrt{4\ell^2 - 2\ell^2}$. $\square$

Two corollaries are worth stating, because they say the error behaves the way one would want.

- The bound vanishes iff $L = c$, that is, iff the athlete travelled in a straight line at any speed over that interval. Speed alone never causes error; only *turning* does. Proposition 5 is the degenerate case.
- It is controlled by curvature and speed together, through $L - c$.

Numbers. A cyclist at 10 m/s with $h = 2$ s covers $L \leq 20$ m. A symmetric right-angle corner at that speed — an aggressive switchback — gives a bound of $10/\sqrt{2} \approx 7.1$ m. Web Mercator ground resolution is

$$\rho(z, \varphi) = \frac{2\pi R \cos \varphi}{256 \cdot 2^z} \quad \text{metres per pixel},$$

which at $z = 14$ and latitude 40° is 7.3 m/px. The worst corner in the worst interval is therefore under one pixel at city zoom. At $z = 17$ it is 0.92 m/px and the same corner is about 8 px — visible, if you zoom to a switchback and look for it. That is the whole cost.

A tangential component also exists and is not bounded by Proposition 6: if the athlete stops inside I , $\tilde{P}$ keeps gliding. It is bounded crudely by $L \leq vh$, it does not move the mark off the track, and it is invisible because a mark's position along the track is not independently observable — only its spacing from its neighbours is, and that is a T -scale quantity, not an h -scale one.

8. Gaps

Recordings contain gaps — a tunnel, a dead battery, a paused watch — across which P interpolates a straight line the athlete did not travel. Gaps are detected as outliers in the inter-sample displacement and carried through resampling as a flag: each node i gets

$$\nu_i = \begin{cases} 0 & \text{if the recorded segment containing } ih \text{ is a gap,} \\ 1 & \text{otherwise,} \end{cases}$$

and a mark is displayed only if both of its bracketing nodes have $\nu = 1$. A mark therefore extinguishes on entering a gap and reappears on the far side, rather than sliding across a fiction.

9. The model, complete

Collecting Definition 2 with Propositions 3 and 4, an activity displays, at phase ϕ , a mark for each $k \in \{0, \dots, \lfloor D/T \rfloor\}$, at

$$\theta_k = (\phi - \alpha) \bmod T + kT, \quad \text{position } \tilde{P}(\theta_k),$$

drawn iff $\theta_k \leq D$ and both bracketing nodes are valid. Per frame, ϕ changes and nothing else does.

Part II — Evaluation on a GPU

10. What a vertex shader can and cannot do

A vertex shader is a function invoked once per vertex, in parallel across thousands of cores, which must write exactly one clip-space position. It has no output list to append to, no visibility of other invocations, and no efficient way to run a data-dependent loop — divergent branches within a warp are serialized. It reads per-vertex *attributes* from a buffer, per-draw *uniforms* shared by every invocation, and *textures* at arbitrary integer indices in constant time (`texelFetch` in GLSL ES 3.00 — an integer-indexed read with no filtering, which is what lets a texture serve as general-purpose memory rather than an image).

Three constraints follow, and the whole of Part I was arranged to satisfy them.

1. **One invocation, one vertex.** The set of marks must be enumerable in advance, with a stable name per mark. That is Proposition 3.
2. **Uniform per-frame input.** Whatever changes each frame must be small and shared. That is Proposition 4.
3. **No search.** Position lookup must be arithmetic. That is §6.

11. Inverting the loop

The Canvas 2D renderer this replaced solved the same equations from the other end. It iterated over *segments* and, for each one spanning $[\theta_a, \theta_b]$, solved Definition 1 for the marks landing inside it:

```
const kLow  = Math.ceil((now - tb) / T)
const kHigh = Math.floor((now - ta) / T)
if (kLow > kHigh) continue // no mark falls in this segment
for (let k = kLow; k <= kHigh; k++) {
  const dt = (now - k * T) - ta
  dotlocs[count * 2] = pax + vx * dt
  dotlocs[count * 2 + 1] = pay + vy * dt
  count++
}
```

This is *segment-major*: the outer loop runs over the track, the inner over marks, and the answer accumulates in a variable-length array. It is the right shape for a CPU, because standing on a segment means already holding the bracket — the search of §6 is free, and never appears.

A vertex shader cannot be organized this way, for the structural reason in constraint 1: there is no array to append to. The loop must be inverted to *mark-major* — “I am mark (a, k) ; where am I?” — and inverting it is exactly what makes the bracket expensive, because an arbitrary θ no longer arrives with its enclosing segment attached. The uniform mesh pays that cost once, on the host, for every θ the shader could ever ask about.

The trade is worth naming: segment-major work is $O(\text{segments in view})$ per frame on the main thread; mark-major work is $O(1)$ on the main thread and $O(\text{marks})$ on hardware built for it.

12. What lives on the GPU

Three resources, each with its own update schedule.

resource	contents	rewritten when
stream texture	every activity's resampled nodes, concatenated	activities load
meta texture	4 texels per activity	selection, colour, view origin, terrain calibration
slot buffer	one (a, k) pair per mark in K	T or the selection changes

Stream texture. One RGBA32F texel per node, holding $(x, y, \text{altitude}, \nu)$ — position relative to the activity's own centre, so the values stay small. Activities are laid end to end, wrapping at a texture width $W = \min(4096, \text{the device's largest texture})$. Sizing is done before writing: if the total exceeds $\min(W^2, 2^{24})$ texels, $h_{\max}$ doubles and the total is recomputed. The second cap is there because node offsets are carried through the shader as binary32, which represents consecutive integers exactly only up to 2^{24} .

Meta texture. Four texels per activity, holding what §5 separated out:

texel	x	y	z	w
0	node offset	node count	$1/h$	α
1	centre x	centre y	px per metre	altitude offset
2	colour r	g	b	a
3	selected	D	—	—

The centre in texel 1 is the activity's own, minus the view origin of §15, which is why the meta texture is rewritten when that origin moves.

Slot buffer. Two uint16s per mark. Its length is $\sum_a (\lfloor D_a/T \rfloor + 1)$ — Proposition 3, summed — and the marks are ordered unselected-first so that a selected activity's marks draw last, on top.

13. The vertex shader

Definition 2 transcribed. One invocation per slot; its entire job is to decide whether this mark exists right now and, if so, where.

```

int a = int(a_slot.x) * 4;
vec4 m0 = fetch(u_meta, a);           // offset, count, 1/h, anchor
vec4 m3 = fetch(u_meta, a + 3);       // selected, duration

float theta = mod(u_phase - m0.w, u_T) + float(a_slot.y) * u_T;
if (theta > m3.y) { clipped(); return; } // Definition 2: theta_k <= D

float f = theta * m0.z;                // §6: theta / h
float i0 = floor(f);
int base = int(m0.x) + int(i0);
if (i0 + 1.0 >= m0.y) { clipped(); return; } // bracket within this activity

vec4 s0 = fetch(u_streams, base);
vec4 s1 = fetch(u_streams, base + 1);
if (s0.w < 0.5 || s1.w < 0.5) { clipped(); return; } // §8: gap

vec4 m1 = fetch(u_meta, a + 1);
vec3 p = mix(s0.xyz, s1.xyz, f - i0); // §6: the interpolant
bool hidden;

```

```
gl_Position = project(p.xy + m1.xy, (p.z + m1.w) * m1.z * u_zScale, hidden);
if (hidden) { clipped(); return; } // §16: behind the globe
```

Line by line against Part I: `mod(u_phase - m0.w, u_T)` is Proposition 4; adding `float(a_slot.y) * u_T` is Definition 2; `theta * m0.z` and the `mix` are §6; the first three early returns are, in order, Definition 2’s constraint, the concatenation boundary, and §8. The fourth has nothing to do with the model: `project` is the map projection, which on a flat map is a single matrix product and on a globe is §16, and a mark on the far side of the globe is dropped the same way.

`clipped()` writes a position outside the clip volume and a point size of zero — the cheapest way to make a vertex disappear, since the rasterizer discards it before any fragment work.

It is worth quantifying how often that happens, because the natural worry about a fixed slot buffer is that it wastes invocations on marks that do not exist. It does not. The buffer allocates $\lfloor D/T \rfloor + 1$ slots per activity (Proposition 3), and §3 computed the number actually displayed as $\lfloor (D - r)/T \rfloor + 1$, which differs from the allocation by at most one. So **at most one slot per activity is dark at any instant** — the trailing one, which is displayed precisely when $r \leq D \bmod T$ and is therefore dark for a fraction $1 - (D \bmod T)/T$ of each period, on average one half. For a 67-minute activity at $T = 60$ that is half a dark slot out of 68, under one percent.

The early returns are therefore *rare*, not common, and the fixed slot buffer is very nearly tight. The remaining culls — the bracket test and the gap test — are rarer still. The horizon test is the exception, and it is the view’s doing rather than the model’s: on a globe it discards roughly the far hemisphere’s share of the marks, which is the same outcome as the flat map’s off-screen marks failing clip-space testing. Uniform control flow is also what the hardware wants: with almost every invocation in a warp taking the same path, the branches cost essentially nothing to divergence.

The alternative, in any case, would be a host-side compaction pass that reproduces the branch on the CPU and uploads only the survivors — which is precisely the per-frame transfer the design exists to avoid, in exchange for eliminating under one percent of the vertex work.

14. Level of detail: what is simplified, and what deliberately is not

The original renderer simplified each track per zoom level, and reasonably so. The present design keeps that for one of its two geometries and refuses it for the other, and the asymmetry is not an oversight.

Paths are still simplified per zoom, exactly as before. Each activity caches, per integer zoom z , an index set produced by Douglas–Peucker (with a radial-distance prefilter) at tolerance 2^{-z} world pixels — which is one *screen* pixel at that zoom, since a zoom-0 world pixel spans 2^z of them. The sets are built lazily off the critical path when a zoom is first visited and cached thereafter, and the path vertex buffer is assembled from the index set alone. Path vertex count is therefore proportional to the on-screen complexity of what is visible, not to the number of recorded points, which is the property that makes a thousand loaded activities affordable.

Two details of the present implementation are worth recording. The zoom used for simplification is $\max(0, \text{round}(z_{\text{ML}} + 1)) + \beta$: the $+1$ reconciles MapLibre’s 512-pixel tile convention with the 256-pixel one the index sets were built around, and $\beta = 1$ when the pitch exceeds 40° , buying one level of extra detail because a pitched view shows foreground ground closer than the centre. And simplification is only half the reduction — the paths are also culled to a padded viewport, and rebuilt at most once per 120 ms during a gesture.

Marks are not simplified at all, and must not be. The stream texture holds every activity at full extent, resampled in *time*; zoom appears nowhere in §6, nor in the slot buffer, nor in the dot shader. Three reasons, in increasing order of importance.

It would not remove any marks. The number of marks is $\lfloor D/T \rfloor + 1$ — a function of duration and the sparsity dial, and of nothing else. A four-hour ride at $T = 60$ contributes 240 marks whether the recording holds 2,000 points or 200,000. Thinning the nodes changes the size of the texture, never the number of things drawn. The concern that motivates the question — that full resolution creates points which collapse into one pixel — is a fact about *path* vertices, where it is real and is handled, and simply does not transfer to marks, which were never per-point objects.

The storage is already bounded, twice. The $h \leq 2$ s mesh is itself an aggressive decimation: a 1 Hz four-hour recording holds 14,400 points and contributes 7,200 nodes. And if the total across everything loaded would exceed the texture cap, $h_{\max}$ doubles until it fits, so the texture has a hard ceiling irrespective of what is loaded. A thousand activities averaging 67 minutes measures at 2.0 M texels, 30.8 MB — comfortably inside it.

Spatial simplification would corrupt the signal. This is the real reason. Douglas–Peucker discards a point when it lies within tolerance of the chord joining its neighbours — a criterion on *shape*, evaluated with no reference to time. For a path, which is a static curve, that is exactly right: the discarded point is by construction invisible, and the drawn figure is unchanged. For a mark, position is a function of time, and discarding a point rewrites that function. The pathological case is an athlete who stops: a minute at a traffic light is a hundred recorded points at one location, all of them within any tolerance of the chord through their neighbours, and all of them therefore deleted. The surviving segment interpolates straight through the intersection at the average speed, and the mark glides across it without pausing. By Proposition 2 the spacing between marks *is* the pace readout, so this does not degrade the picture’s accuracy at the margin — it silently falsifies the one quantity the animation exists to display. Uniform resampling in time is immune by construction: it preserves P as a function, and its error is bounded by Proposition 6 in terms of the athlete’s own motion, not in terms of a screen tolerance.

The right summary is that the two geometries are simplified along the axis each one lives on. A path is a shape, so it is decimated in space, against a screen tolerance, per zoom. A mark is a clock reading, so it is decimated in time, against a duration, once.

What is genuinely not handled. Two things, and both are open.

- Marks are not view-culled. The slot buffer is built over *all* loaded activities, not the in-view ones, so every mark of every activity is submitted every frame and off-screen ones are rejected by clip-space testing (or, on a globe, by the horizon test of §16). This is a deliberate simplification — it keeps the slot buffer independent of the view, so panning uploads nothing — and it holds because a culled vertex costs only its own invocation, which is the cheap resource here. It would stop holding at a load where vertex throughput, rather than main-thread time, became the constraint. Nothing measured so far is close.
- At low zoom, many activities occupying the same few pixels are drawn over one another with nothing to indicate how many are present. Mark size grows with zoom as $2^{0.15(z+1-4)}$, floored at 0.25 px, which mitigates the symptom in both directions — a single size cannot serve a view where an entire ride is four pixels wide and one where consecutive marks are far apart — but it is a tuned constant standing in for the thing actually wanted, which is clustering with an explicit density readout. Overdraw itself is bounded by sprite area rather than mark count, so this is a legibility problem and not a performance one. Part of it is now addressed outside the GL layer: at map zoom 8 and below, an activity smaller than 10 px on screen gets a marker, and markers within 32 px of one another merge into one. That answers “is anything here?”, which the marks alone could not at that scale; it still does not say *how much*, since a marker carries no count.

15. Precision

Both shaders receive positions relative to a nearby origin rather than absolute coordinates, and the margin this buys can be computed exactly.

Activities are stored in zoom-0 world pixels: the projection maps the whole Earth into $[0, 256)^2$, so an absolute coordinate has magnitude up to 2^8 . At zoom z the map scale is 2^z , so one screen pixel is 2^{-z} world pixels. A binary32 significand is 24 bits, so the representable spacing near 2^8 is

$$2^8 \cdot 2^{-23} = 2^{-15} \text{ world px} = 2^{z-15} \text{ screen px},$$

which at $z = 18$ is *eight screen pixels*. An absolute coordinate at street zoom cannot address a pixel at all, and panning would move the marks in visible jumps.

The origin is the map centre rounded to a whole world pixel — exactly representable, so the subtraction contributes no error of its own — which bounds the relative coordinate by 2^{-1} . The spacing there is

$$2^{-1} \cdot 2^{-23} = 2^{-24} \text{ world px} = 2^{z-24} \text{ screen px},$$

a factor of $2^9 = 512$ better, and $1/64$ of a pixel at $z = 18$. The expression also states the design's limit plainly: rounding error reaches one screen pixel at $z = 24$, which is past the deepest zoom the map offers.

Note that this is achieved by rounding the origin *coarsely*. A finer origin would shrink the relative coordinates further, but it is unnecessary — 2^9 is already ample — and an origin that is not exactly representable would reintroduce error in the subtraction it is meant to remove.

The origin is folded back in by a matrix multiplication performed on the host in binary64, once per frame rather than once per vertex, where full precision is free. This is the same manoeuvre as §5(b) — reduce in double precision, narrow afterwards — applied to space rather than to time, and the two together are what make a single-precision pipeline viable over an epoch-scale clock and a planet-scale coordinate system. §16 is the one place the pipeline goes back to absolute coordinates, and it does so only where the scale makes that safe.

16. The globe

MapLibre can draw the map as a sphere, and the layer follows it. Nothing in Part I changes: Definition 2 takes no argument from the view (§5(c)), and a projection is part of the view. What changes is the last line of each shader, the map from a world position to clip space, which becomes a function `project` shared by the path and mark shaders.

The sphere. A point at Mercator coordinates $(x, y) \in [0, 1)^2$ has longitude $\lambda = 2\pi x - \pi$ and Mercator ordinate $\psi = \pi - 2\pi y$, from which latitude is the Gudermannian, $\varphi = \text{gd } \psi$. The shader never forms φ . With $t = e^\psi$,

$$\cos \varphi = \text{sech } \psi = \frac{2t}{t^2 + 1}, \quad \sin \varphi = \tanh \psi = \frac{t^2 - 1}{t^2 + 1},$$

so one exponential and two trigonometric calls on λ give the point on the unit sphere,

$$S = (\sin \lambda \cos \varphi, \sin \varphi, \cos \lambda \cos \varphi).$$

This is MapLibre's own `projectToSphere`, reproduced so that marks and the basemap agree to the arithmetic.

Height. Terrain lifts a point by ζ world pixels. One Mercator unit at latitude φ spans $2\pi R \cos \varphi$ of ground, so ζ is a fraction $(\zeta/256) 2\pi \cos \varphi$ of the radius, and the lifted point is

$$E = S \left(1 + \frac{\zeta}{256} 2\pi \cos \varphi \right).$$

The $\cos \varphi$ is the Mercator scale factor undone: the flat map exaggerates heights toward the poles exactly as it exaggerates lengths, and the sphere must not.

The horizon. MapLibre supplies a plane $(\mathbf{n}, w)$ through the horizon circle, and a point is on the visible side iff $\langle S, \mathbf{n} \rangle + w \geq 0$. A mark that fails is dropped (§13); a path segment is dropped if either end fails. Without the test, the far hemisphere’s tracks draw through the Earth, since the layer does not write depth.

The transition. MapLibre does not switch projections at a zoom; it blends them. Its globeness g runs from 1 when zoomed out to 0 between map zooms 11 and 12, and the layer returns

$$\text{clip} = (1 - g) M_{\text{flat}} p + g M_{\text{globe}} E,$$

matching the basemap through the blend. At $g = 0$ the sphere is not computed at all.

Precision. The sphere needs absolute coordinates: the shader re-forms $(x, y) = \text{origin} + p/256$ in binary32 before taking the exponential. By the reckoning of §15, a Mercator coordinate below 1 has spacing 2^{-24} , which is 2^{-16} world pixels, or 2^{z-16} screen pixels at zoom z . The sphere is in use only below map zoom 12, which is $z = 13$ in the 256-pixel convention of §15, where that is an eighth of a pixel; zoomed out, where the globe is actually a globe, it is thousands of times smaller. The transcendental functions add error of their own, which GLSL ES does not bound; the figure above is the part that can be computed. Handing the sphere off at $g = 0$ is what keeps this safe: street zoom, where §15’s margin is needed, never touches the absolute path.

17. Cost

The per-frame transfer is the camera matrix, a handful of scalars, and ϕ : of order 100 bytes, of which the animation proper is four; a globe adds a second matrix and the horizon plane. Steady state is six draw calls regardless of load: the marks’ shadow silhouette into a half-resolution buffer, two separable blur passes over it, the paths, the shadow composited onto the map, and the marks. Only the first and last scale with the number of marks.

Measured on the live site with Chrome’s own counters on 17 September, a day before the shadow took its present one-silhouette form, main-thread JavaScript per frame is 0.563 ms with 60 activities loaded and 0.485 ms with 1,000 (68,240 marks drawn per frame, 2.0 M texels of stream texture). Seventeen times the load, no measurable change; zooming four levels moves it by 0.006 ms. The one term that does respond is dragging, 0.25 ms dearer at the larger load — that is the path rebuild of §14, the only per-view CPU work left.

The comparison worth making is asymptotic rather than constant-factor. The segment-major loop of §11 measures at about 5 ns per in-view segment plus 5 ns per mark emitted, so its cost is $\Theta(\text{track in view})$ per frame; at the thousand-activity load that is milliseconds, and it grows with everything the user loads. The mark-major cost is $\Theta(1)$ on the main thread. Drawing less often — the 2020 renderer gated at 25 fps — divides the former by a constant and moves the cliff; it does not remove it.

Colophon

Heatflask is at <https://www.heatflask.com>; its source is AGPL-3.0-or-later at <https://github.com/ebrensi/heatflask>. The model of Part I dates to the original Canvas 2D renderer (2016–2020); the WebGL2 realization of Part II landed in September 2026.

Open: a licence for this text. AGPL governs the source, not the prose. If the purpose is to establish authorship of the method while leaving others free to build on it, CC BY 4.0 is the conventional choice — attribution required, derivatives permitted. A DOI via Zenodo would give it a citable, timestamped identifier; that is the cheapest thing that makes “published in September 2026” independently verifiable.